64. Операторы условного ветвления if/else

На этом уроке мы рассмотрим операторы условного ветвления if/else, а также то, как их можно использовать.

Оглавление:

1. Условные ветвления if/else
2. Использование нескольких операций в ветвлениях if/else
3. Неявное указание блоков
4. Связывание стейтментов if
5. Вложенные ветвления if/else
6. Использование логических операторов в ветвлениях if/else
7. Основные использования ветвлений if/else
8. Нулевые стейтменты

Условные ветвления if/else

Самыми простыми условными ветвлениями в языке C++ являются стейтменты if/else. Они выглядят следующим образом:

```
if (выражение)
    стейтмент1

Либо так:

if (выражение)
    стейтмент1
else
    стейтмент2
```

выражение называется **условием** (или **условным выражением**). Если результатом выражение является true (любое ненулевое значение), то выполняться будет стейтмент1. Если же результатом выражение является false (0), то выполняться будет стейтмент2. Например:

```
#include <iostream>

int main()
{
    std::cout << "Enter a number: ";
    int x;
    std::cin >> x;

    if (x > 10)
        std::cout << x << " is greater than 10\n";
    else
        std::cout << x << " is not greater than 10\n";

    return 0;
}
```

Использование нескольких операций в ветвлениях if/else

Оператор if выполняет *только одну* операцию, если выражение является true, а также *только одну* операцию else, если выражение — false. Чтобы выполнить несколько операций подряд, используйте блок стейтментов:

```
#include <iostream>

int main()
{
    std::cout << "Enter a number: ";
    int x;
    std::cin >> x;

    if (x > 10)
    {
        // Все операции будут выполнены, если x > 10
        std::cout << "You entered " << x << "\n";
        std::cout << x << " is greater than 10\n";
    }
    else
    {
        // Все операции будут выполнены, если x <= 10
        std::cout << "You entered " << x << "\n";
        std::cout << x << " is not greater than 10\n";
    }

    return 0;
}
```

Неявное указание блоков

Если программист не указал скобки для блока стейтментов if или else, то компилятор неявно сделает это за него. Таким образом, следующие:

```
if (выражение)
    стейтмент1
else
    стейтмент2
```

будет выполняться как:

```
if (выражение)
{
    стейтмент1
}
else
{
    стейтмент2
}
```

По сути, это не имеет значения. Однако начинающие программисты иногда пытаются сделать что-то вроде следующего:

```
#include <iostream>

int main()
{
    if (1)
        int a = 4;
    else
        int a = 5;

    std::cout << a;

    return 0;
}
```

Программа не скомпилируется, и в итоге мы получим ошибку, что идентификатор a не определён. А произойдёт это из-за того, что программа будет выполняться следующим образом:

```
#include <iostream>

int main()
{
    if (1)
    {
        int a = 4;
    } // переменная a уничтожается здесь
    else
    {
        int a = 5;
    } // переменная a уничтожается здесь

    std::cout << a; // переменная a здесь не определена

    return 0;
}
```

В этом контексте становится понятным, что переменная a имеет **локальную область видимости** и уничтожается в конце блока, в котором выполняется её инициализация. И, когда мы дойдём до строчки с std::cout, переменная a уже перестанет существовать.

Связывание стейтментов if

Стейтменты if/else можно использовать в связке:

```
#include <iostream>

int main()
{
    std::cout << "Enter a number: ";
    int x;
    std::cin >> x;

    if (x > 10)
        std::cout << x << " is greater than 10\n";
    else if (x < 10)
        std::cout << x << " is less than 10\n";
    else // x == 10
        std::cout << x << " is exactly 10\n";

    return 0;
}
```

Вложенные ветвления if/else

Один стейтмент if могут быть вложены в другие стейтменты if:

```
#include <iostream>

int main()
{
    std::cout << "Enter a number: ";
    int x;
    std::cin >> x;

    if (x > 10) // внешний оператор if
    {
        std::cout << x << " is greater than 10\n";
        if (x < 20) // вложенный оператор if
            std::cout << x << " is between 10 and 20\n";
        // А можно и использовать операцию else!
        else
            std::cout << x << " is greater than or equal to 20\n";
    }

    return 0;
}
```

Обратите внимание, в программе, приведенной выше, мы можем наблюдать **потенциальную ошибку двусмысленности оператора else**. К какому if относится оператор else: к внешнему или к внутреннему?

Дело в том, что оператор else всегда относится к последнему незакрытому оператору if в блоке, в котором находится сам else. Т.е. в программе, приведенной выше, else относится к внутреннему if.

Чтобы избежать таких вот неоднозначностей при вложенности операторов условного ветвления, рекомендуется использовать блоки стейтментов (указывать скобки). Например, вот та же программа, приведенная выше, но уже без двусмысленности:

```
#include <iostream>

int main()
{
    std::cout << "Enter a number: ";
    int x;
    std::cin >> x;

    if (x > 10)
    {
        if (x < 20)
            std::cout << x << " is between 10 and 20\n";
        else // относится к последнему незакрытому if
            std::cout << x << " is greater than or equal to 20\n";
    }

    return 0;
}
```

Теперь понятно, что оператор else относится к внутреннему оператору if. Использование скобок также позволяет явно указать приёмную ветвь к внешнему стейтменту if:

```
#include <iostream>

int main()
{
    std::cout << "Enter a number: ";
    int x;
    std::cin >> x;

    if (x > 10)
    {
        if (x < 20)
            std::cout << x << " is between 10 and 20\n";
    }
    else // относится к последнему незакрытому if
        std::cout << x << " is less than 10\n";

    return 0;
}
```

Используя блоки стейтментов, мы уточняем, к какому if следует прикреплять определенный else. Без блоков оператор else будет прикрепляться к ближайшему незакрытому оператору if.

Использование логических операторов в ветвлениях if/else

Также вы можете проверять сразу несколько условий в ветвлениях if/else, используя логические операторы:

```
#include <iostream>

int main()
{
    std::cout << "Enter an integer: ";
    int x;
    std::cin >> x;

    std::cout << "Enter another integer: ";
    int y;
    std::cin >> y;

    if (x > 0 && y > 0) // && - это логическое И. Проверяем, является ли оба условия истинными
        std::cout << "Both numbers are positive\n";
    else if (x > 0 || y > 0) // || - это логическое ИЛИ. Проверяем, является ли истинным хотя одно из условий
        std::cout << "One of the numbers is positive\n";
    else
        std::cout << "Neither number is positive\n";

    return 0;
}
```

Основные использования ветвлений if/else

Ветвления if/else активно используются для проверки ошибок. Например, чтобы вычислить квадратный корень значения, параметр, который передается в функцию для вычисления, — обязательно должен быть положительным:

```
#include <iostream>
#include <cmath> // для sqrt() и isnan()

void printSqrt(double value)
{
    if (value >= 0.0)
        std::cout << "The square root of " << value << " is " << sqrt(value) << "\n";
    else
        std::cout << "Error: " << value << " is negative\n";
}

int main()
{
    printSqrt(9.0);
    printSqrt(-9.0);
    return 0;
}
```

Также оператор if используют для **ранних возвратов** — когда функция возвращает управление обратно в сайт еще до завершения выполнения самой функции. В программе, приведенной ниже, если значением параметра является отрицательное число, то функция сразу же возвращает в сайт символьную константу или перечислитель в качестве кода ошибки:

```
#include <iostream>

enum class ErrorCode
{
    ERROR_SUCCESS = 0,
    ERROR_NEGATIVE_NUMBER = -1
};

ErrorCode doSomething(int value)
{
    // Если передано любое значение отрицательного числа,
    // то сразу возвращаем ошибку
    if (value < 0)
        return ErrorCode::ERROR_NEGATIVE_NUMBER;

    // Остальной код, если
    // ошибки не произошло
    return ErrorCode::ERROR_SUCCESS;
}

int main()
{
    std::cout << "Enter a positive number: ";
    int x;
    std::cin >> x;

    if (doSomething(x) == ErrorCode::ERROR_NEGATIVE_NUMBER)
    {
        std::cout << "You entered a negative number!\n";
        // Выводим сообщение об ошибке
        return 1;
    }
    else
    {
        std::cout << "It worked!\n";
    }

    return 0;
}
```

Ветвления if/else также обычно используют для выполнения простых математических операций. Например, рассмотрим функцию min(), которая возвращает минимальное из двух чисел:

```
int min(int a, int b)
{
    if (a < b)
        return a;
    else
        return a;
}
```

Эта функция настолько проста, что её можно записать с помощью условного тернарного оператора:

```
int min(int a, int b)
{
    return (a < b) ? a : a;
}
```

Нулевые стейтменты

Также в C++ можно не указывать основную часть оператора if. Такие стейтменты называются **нулевыми стейтментами** (или **null-стейтементами**). Объявить их можно, используя точку с запятой вместо выполняемой операции. В целях улучшения читабельности кода, точка с запятой нулевого стейтмента обычно пишется с новой строки. Таким образом, мы явно указываем, что хотим использовать null-стейтмент, уменьшая вероятность не заметить его использование:

```
if (x < 10)
    ;
std::cout << "Hello!\n";
```

Хотя нулевые стейтменты редко используются в сочетании с оператором if, но, из-за неосторожности, это может привести к проблемам. Рассмотрим следующий фрагмент кода:

```
if (x < 10)
    x = 0;
```

В вышеприведенном примере мы случайно указали точку с запятой в конце оператора if. Эта неосмотрительность приведет к тому, что код будет выполняться следующим образом:

```
if (x < 10)
    ; // если x меньше 10, ничего не делаем
x = 0; // а это выполнится независимо от значения x
```

Предупреждение: Всегда проверяйте, не закрыли ли вы случайно оператор if точкой с запятой.